Document Scanner Implementation

Week 03 laboratory report
Liu Yixuan | 23 September 2026

Introduction

The objective is to locate a document in a photograph and transform its four corners into an axis-aligned rectangle. The four required stages are implemented using OpenCV: Gaussian denoising, Canny edge detection, quadrilateral selection and perspective warping. One shared parameter set processes all three supplied images successfully. The second image is rectified using its printed inner border; its full wooden outline is not recovered.

Implementation

1 Preprocessing. Convert the original BGR image to grayscale, then call `cv2.GaussianBlur`. The original color image is retained for the final warp, so smoothing affects detection without directly blurring the saved document.

2 Edge detection. Apply `cv2.Canny`, then one 5 x 5 morphological closing operation. The added closing step repairs short interruptions in a rectangular border before contour extraction.

3 Contour filtering. Use external contours with chain approximation. Reject contours smaller than 5% of the image, compute each perimeter, and approximate polygons with a tolerance of 2% of that perimeter. Keep convex quadrilaterals and select the largest by original contour area. Return the four vertices as a 4 x 2 array.

4 Perspective correction. Order corners as top-left, top-right, bottom-right and bottom-left. Compute the output width and height from the longer opposing edges, then call `cv2.getPerspectiveTransform` and `cv2.warpPerspective` using matching float32 point arrays.

Final parameters

Stage	Setting used for every image
Gaussian smoothing	9 x 9 kernel; sigma = 2.0
Canny and gap repair	Low/high thresholds = 50/150; one 5 x 5 rectangular closing
Contour selection	Area at least 0.05 x image area; epsilon = 0.02 x perimeter; convex; four vertices
Perspective warp	Four point homography; OpenCV default linear interpolation

Parameter Evaluation and Validation

The following runs use the supplied images at their original resolutions. Canny thresholds (50/150), minimum area (5%) and approximation tolerance (2%) are fixed. A pass means a valid quadrilateral was found and warped; it is not a ground-truth accuracy measurement.

Run	Gaussian	Closing	Case 1	Case 2	Case 3
A	5 x 5 sigma 1.4	None	Fail	Fail	Pass
B	9 x 9 sigma 2.0	None	Pass	Fail	Pass
C	9 x 9 sigma 2.0	3 x 3	Pass	Pass	Pass
D selected	9 x 9 sigma 2.0	5 x 5	Pass	Pass	Pass
E	15 x 15 sigma 3.0	5 x 5	Pass	Fail	Pass

Observed tradeoff. Moderate smoothing recovers Case 1, but Case 2 also needs gap repair. With 3 x 3 closing its contour covers 39.46% of the image; 5 x 5 closing selects the outside edge of the printed frame and covers 40.35%. The stronger 15 x 15, sigma 3.0 blur again loses a valid Case 2 quadrilateral. Run D therefore provides the preferred result across these three images.

Threshold reasoning. Canny rejects weak responses and retains intermediate responses connected to strong edges. Lower thresholds admit noise; higher thresholds can break boundaries. The 1:3 ratio is a starting point. A small polygon tolerance retains extra vertices; a large tolerance can misclassify unrelated shapes as quads.

Verification and corrections

All **10 checks passed**: three supplied-image PNG round trips; a colored-corner warp under all 24 vertex permutations; diamond ordering; degenerate-point rejection; blank-image failure and state reset; debug display; execution from another working directory; and unsuccessful-save exit status.

The result is now assigned before debug display. Paths are relative to the script, the output directory is created automatically, and failed saves produce a nonzero exit status. Detection uses no case-specific coordinates or parameter overrides.

Limitations

The largest-quadrilateral rule can select an internal frame or a background object. Case 2 demonstrates this explicitly. Fixed thresholds and a 5% minimum area may fail for dim scenes, small documents or broken boundaries. The longer-edge sizing rule removes perspective skew but does not recover physical paper proportions, especially for Case 1. No OCR, lighting correction, lens calibration or curved-page flattening is performed.

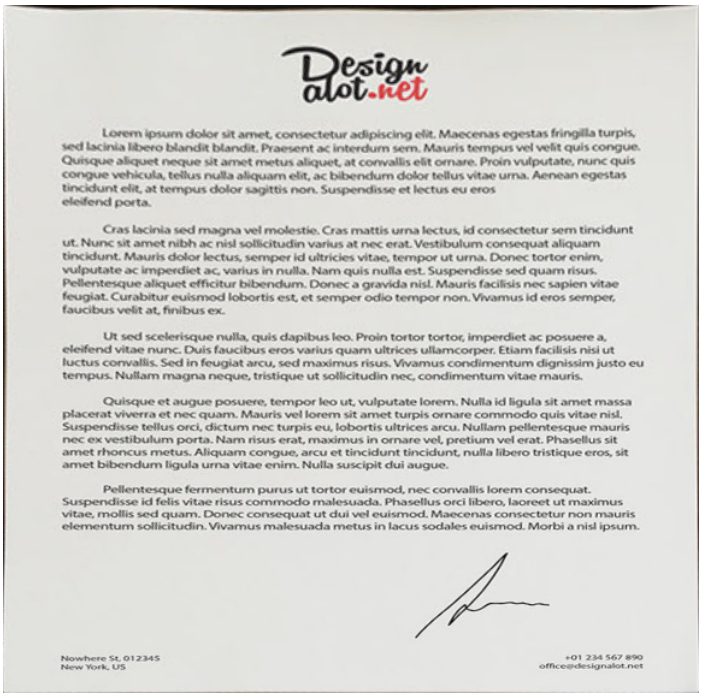
Case 1 Desktop Document

Input asset/case1.png (1024 x 768 pixels) **Output** output/result_1.png (578 x 572 pixels)

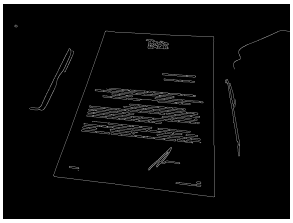
Original with detected contour



Saved rectified output



Canny edges



After closing

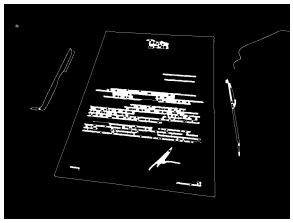


Figure 1. Green markers 1-4 indicate the ordered corners (top-left, top-right, bottom-right, bottom-left). The right panel displays the saved PNG without additional image enhancement.

Result analysis

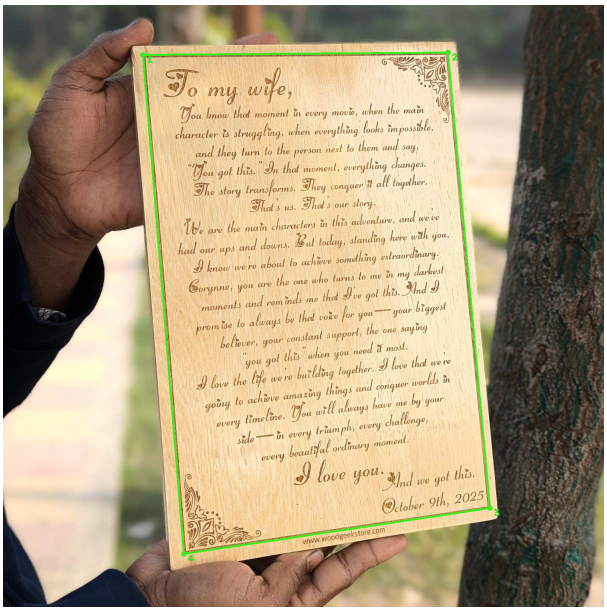
The selected contour follows the paper boundary and removes the surrounding desk, pen and other objects. The text and signature are preserved, and the page edges are aligned with the output axes. A narrow dark strip remains near the top edge. The output is nearly square because its dimensions follow projected edge lengths; the physical page aspect ratio cannot be established from this rule.

Detected corners (366, 96); (751, 116); (754, 688); (180, 613)
Contour area 265,111.0 pixels squared (33.71% of the input image).

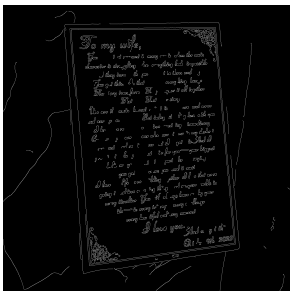
Case 2 Wooden Message Board

Input asset/case2.png (1500 x 1500 pixels) **Output** output/result_2.png (772 x 1247 pixels)

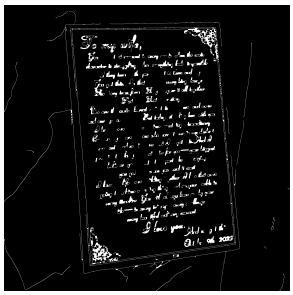
Original with detected contour



Canny edges



After closing



Saved rectified output

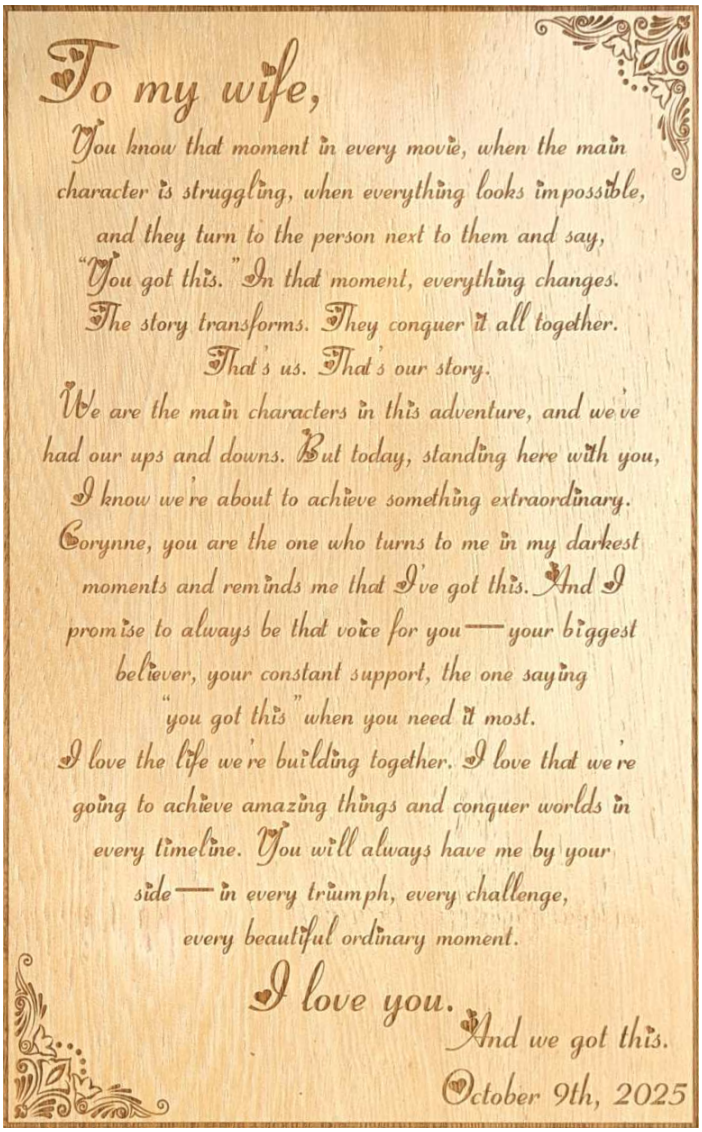


Figure 2. Green markers 1-4 indicate the ordered corners (top-left, top-right, bottom-right, bottom-left). The right panel displays the saved PNG without additional image enhancement.

Result analysis

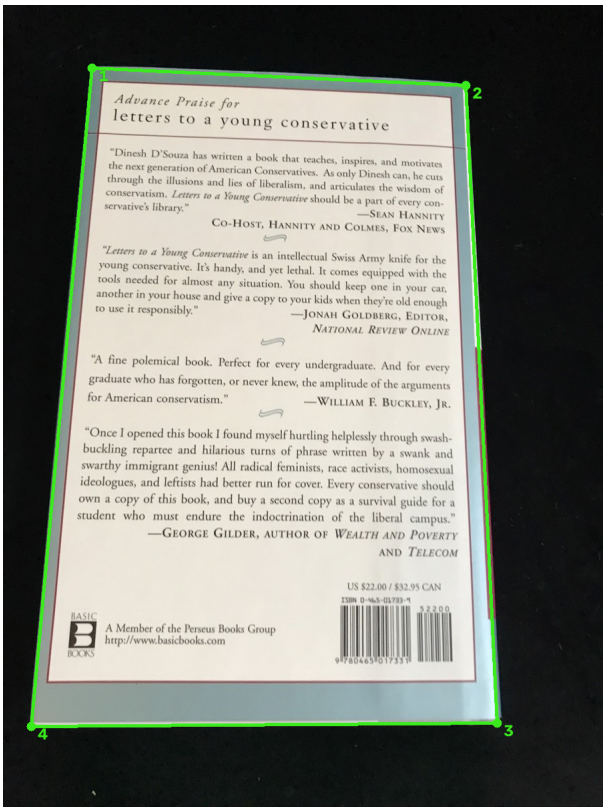
Hands, wood texture and background structures make the outside silhouette difficult to close. The selected quadrilateral follows the printed inner frame. The message and decorative motifs remain visible after correction, while the outer wooden margin and the small website label below the frame are excluded. This is a useful content crop, but it is not a complete recovery of the board's physical boundary.

Detected corners (349, 124); (1110, 118); (1215, 1253); (451, 1367)
Contour area 907,960.5 pixels squared (40.35% of the input image).

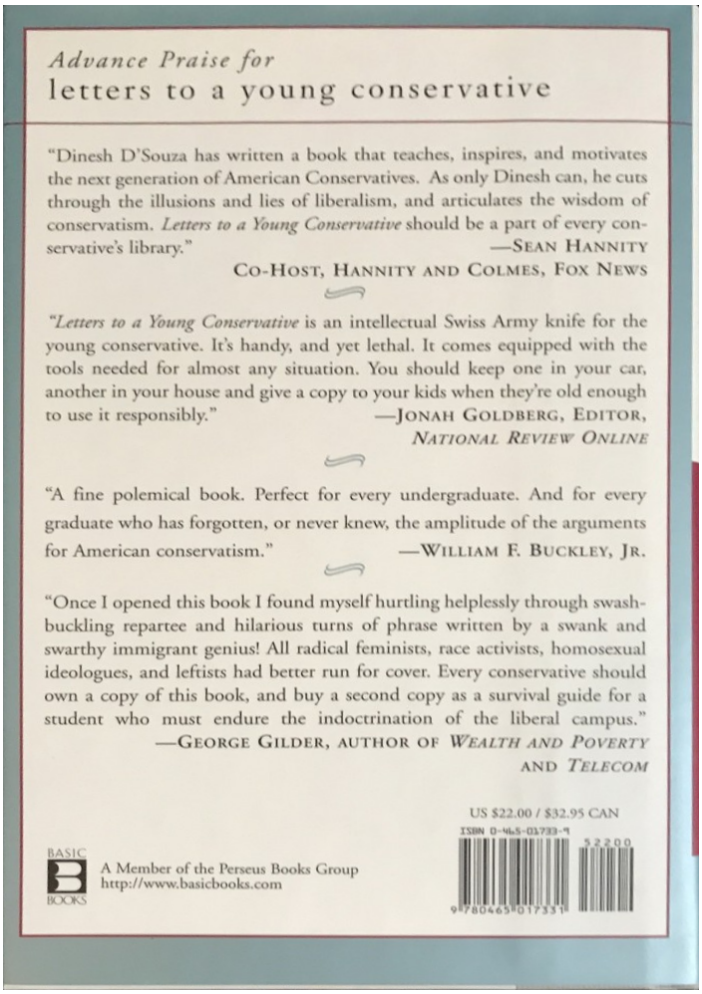
Case 3 Book Back Cover

Input asset/case3.png (899 x 1200 pixels) **Output** output/result_3.png (698 x 989 pixels)

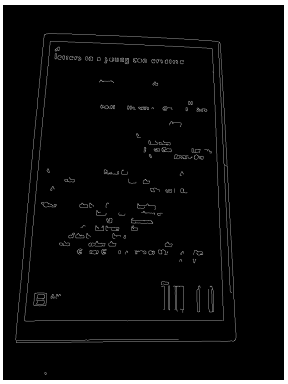
Original with detected contour



Saved rectified output



Canny edges



After closing

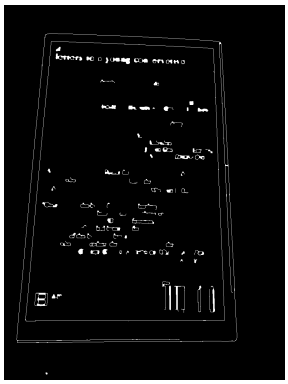


Figure 3. Green markers 1-4 indicate the ordered corners (top-left, top-right, bottom-right, bottom-left). The right panel displays the saved PNG without additional image enhancement.

Result analysis

The selected contour tracks the book's outer cover rather than the text blocks or barcode. The black background is largely removed, and the cover becomes a rectangular frontal view. Slight edge curvature and a narrow background sliver remain because a single homography models a plane and four straight edges.

Detected corners (133, 94); (693, 120); (740, 1074); (42, 1079)
Contour area 620,886.5 pixels squared (57.55% of the input image).

Execution Evidence and Reproduction

The program completed all three test cases with exit code 0. The full standard output, including saved filenames, dimensions and ordered corners, is reproduced in the screenshot below. This is a browser-rendered transcript of the actual process output, rather than a native terminal-window capture.

Week 03 execution transcript

Captured directly from the scanner process, with all three cases and the exit status.

```
PS lab_week03> python student_document_scanner.py
Document Scanner Test
========================================
Gaussian: 9x9, sigma=2.0 | Canny: 50/150
Closing: 5x5 | Min area: 5% | Epsilon: 2%

Test Case 1: asset/case1.png
[OK] Processing successful, result saved to: output/result_1.png
  Output dimensions: (572, 578, 3)
  Corners (TL/TR/BR/BL): [[366, 96], [751, 116], [754, 688], [180, 613]]

Test Case 2: asset/case2.png
[OK] Processing successful, result saved to: output/result_2.png
  Output dimensions: (1247, 772, 3)
  Corners (TL/TR/BR/BL): [[349, 124], [1110, 118], [1215, 1253], [451, 1367]]

Test Case 3: asset/case3.png
[OK] Processing successful, result saved to: output/result_3.png
  Output dimensions: (989, 698, 3)
  Corners (TL/TR/BR/BL): [[133, 94], [693, 120], [740, 1074], [42, 1079]]

========================================
Completed: 3/3 test cases passed.

Process exit code: 0
```

Browser-rendered transcript of the actual run; original text retained in terminal_output.txt.

Reproducing the run

Place the submitted script beside the course-provided `asset/` directory. Install OpenCV, NumPy and Matplotlib, then run `python student_document_scanner.py`. The script creates `output/` automatically. Use `--show-intermediate` to display the five processing stages.

Verified environment Python 3.12.14; OpenCV 5.0.0; NumPy 2.5.2; Matplotlib 3.11.2. The environment uses opencv-python-headless; visualization is provided by Matplotlib.

Conclusion

The completed pipeline produces three readable, rectangular document views. The experiments show that edge continuity and contour selection govern success: moderate denoising and a small closing operation allow one parameter set to handle the supplied scenes. Recovering the complete board outline and true document proportions would require additional geometric or segmentation cues.

Course sources LAB_README.md; Submission.md; edge_detection_tutorial.ipynb. Experiments and figures were generated from the supplied assets and the completed script.